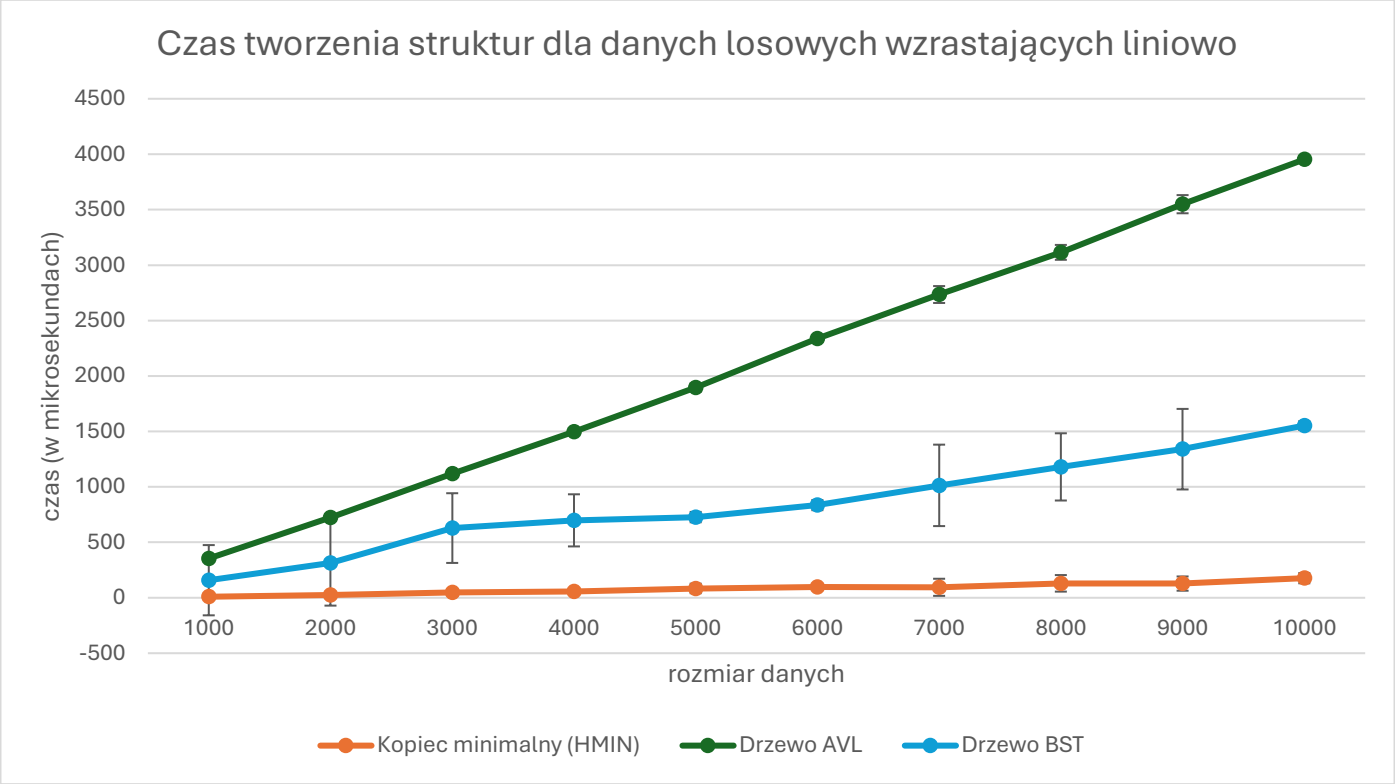
Złożone struktury danych

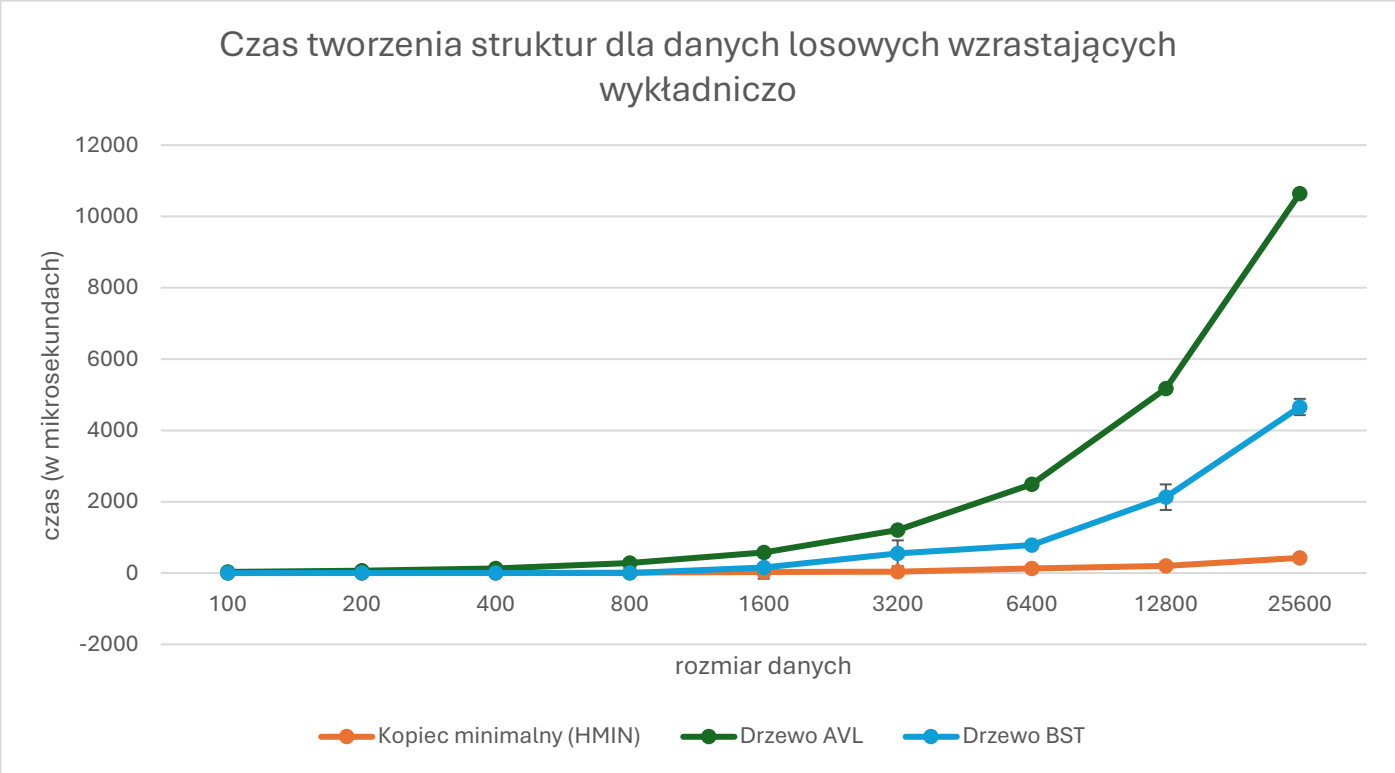
Maria Linkowska 167906, Sara Rachmańczuk 167898

1. **Celem** zadania było zaimplementowanie oraz przeprowadzenie analizy porównawczej trzech dynamicznych struktur danych: drzewa przeszukiwań binarnych (BST), drzewa AVL oraz kopca minimalnego (HMIN). Badania koncentrowały się na analizie czasu wykonywania kluczowych operacji (tworzenie struktury, wyszukiwanie elementu, równoważenie) w zależności od rozmiaru danych (n) oraz ich początkowego uporządkowania (losowe, rosnące, malejące).
2. **Struktury**:
    a. **Drzewo BST konstruowane metodą FCFS** – polega na wstawianiu kolejnych elementów w taki sposób by zachować jego uporządkowaną strukturę. Lewe poddrzewo każdego węzła zawiera wyłącznie elementy o kluczach mniejszych niż klucz węzła, a prawe poddrzewo – elementy o kluczach większych niż klucz węzła. Węzły, oprócz klucza, przechowują wskaźniki na swojego lewego i prawego syna oraz na swojego ojca.
    b. **Drzewo AVL konstruowane metodą wykorzystującą bisekcję** – w tej implementacji tworzone jest z uprzednio posortowanego ciągu danych przy użyciu rekurencyjnego wyboru elementu środkowego (bisekcji). Dzięki temu struktura od początku jest optymalnie zrównoważona, a różnica wysokości lewego i prawego poddrzewa każdego węzła wynosi co najwyżej 1. Węzły, podobnie jak w BST, przechowują klucz oraz wskaźniki na lewego syna, prawego syna i ojca, co pozwala na efektywne przeszukiwanie i poruszanie się po strukturze. Do posortowania danych przed konstrukcją drzewa wykorzystano algorytm Merge Sort o złożoności O(n logn).
    c. **Kopiec minimalny (HMIN) na tablicy jednowymiarowej** – w tej strukturze zachowana jest relacja porządku częściowego: klucz każdego węzła jest mniejszy lub równy kluczom jego synów. Dzięki reprezentacji tablicowej, powiązania między ojcem a synami wyliczane są arytmetycznie na podstawie indeksów, co eliminuje potrzebę przechowywania wskaźników. Korzeń drzewa (element minimalny) zawsze znajduje się na początku tablicy, a cała struktura tworzy drzewo binarne niemal pełne, wypełniane poziomami od lewej do prawej.
3. Testy wykonano dla danych wzrastających liniowo od 1 000 do 10 000 (skok co 1000) oraz dla danych wzrastających wykładniczo od 100 do 25 600. Wyniki zostały pobrane z 10 prób by zminimalizować błąd pomiarowy oraz badania zostały przeprowadzone dla 3 rodzajów danych:
    a. Losowych,
    b. Malejących,
    c. Rosnących.

    Ze względu na wysoką efektywność implementacji oraz krótki czas wykonywania pojedynczych operacji, wyniki pomiarów zaprezentowano w **mikrosekundach**. Pozwoliło to na uniknięcie błędów zaokrągleń do zera i umożliwiło precyzyjne przedstawienie różnic między strukturami.

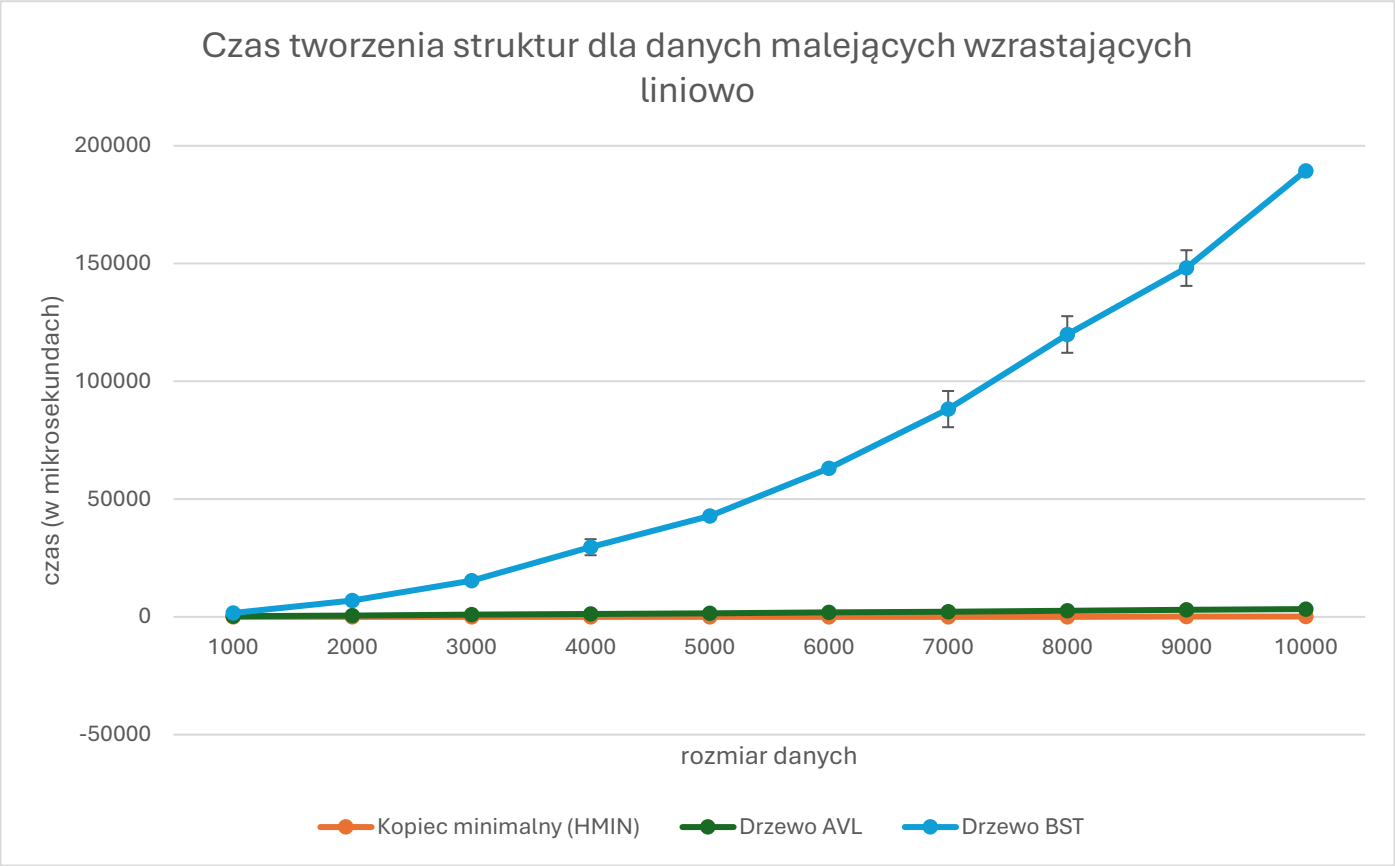
4. **Analiza wyników** – dla każdego typu danych wymienionych w punkcie 3 porównanie czasu tworzenia struktur:
    a. Dla danych losowych:

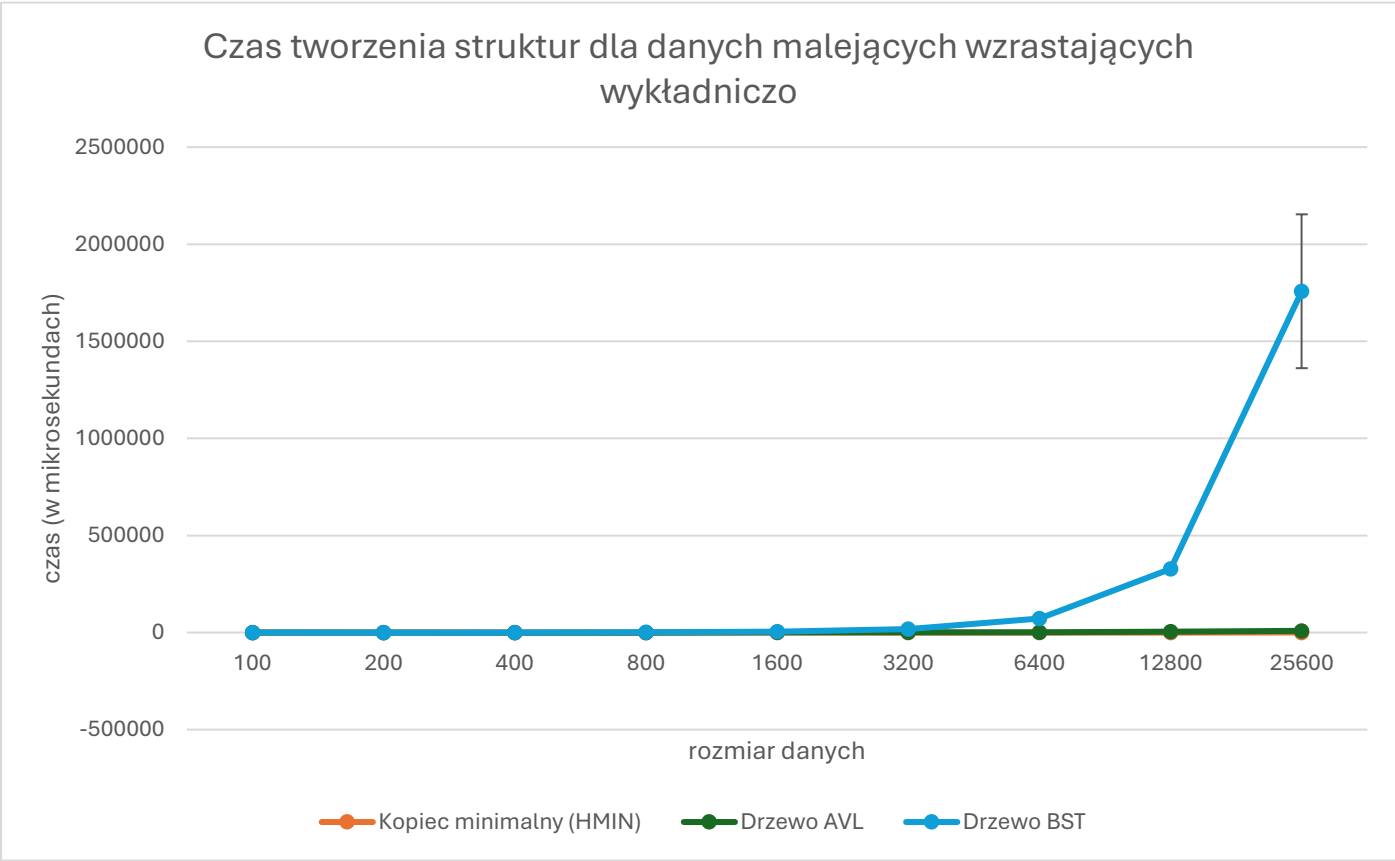




Dla danych losowych różnice pomiędzy tworzeniem struktur są najmniejsze, ale nadal wyraźnie widoczne przy dużym n. Na obu wykresach wyraźnie widać, że kopiec minimalny (HMIN) jest zdecydowanie najszybszy, dzięki prostej implementacji tablicowej i braku konieczności balansowania wskaźników. Następne w kolejności mamy drzewo BST, które przy danych losowych jest naturalnie dość dobrze zrównoważone, co daje złożoność bliską O(n logn). Za to budowanie drzewa AVL zajęło najdłużej mimo tej samej klasy złożoności co drzewo BST. Ten narzut czasowy wynika z metody bisekcji i konieczności sortowania danych przed budową.

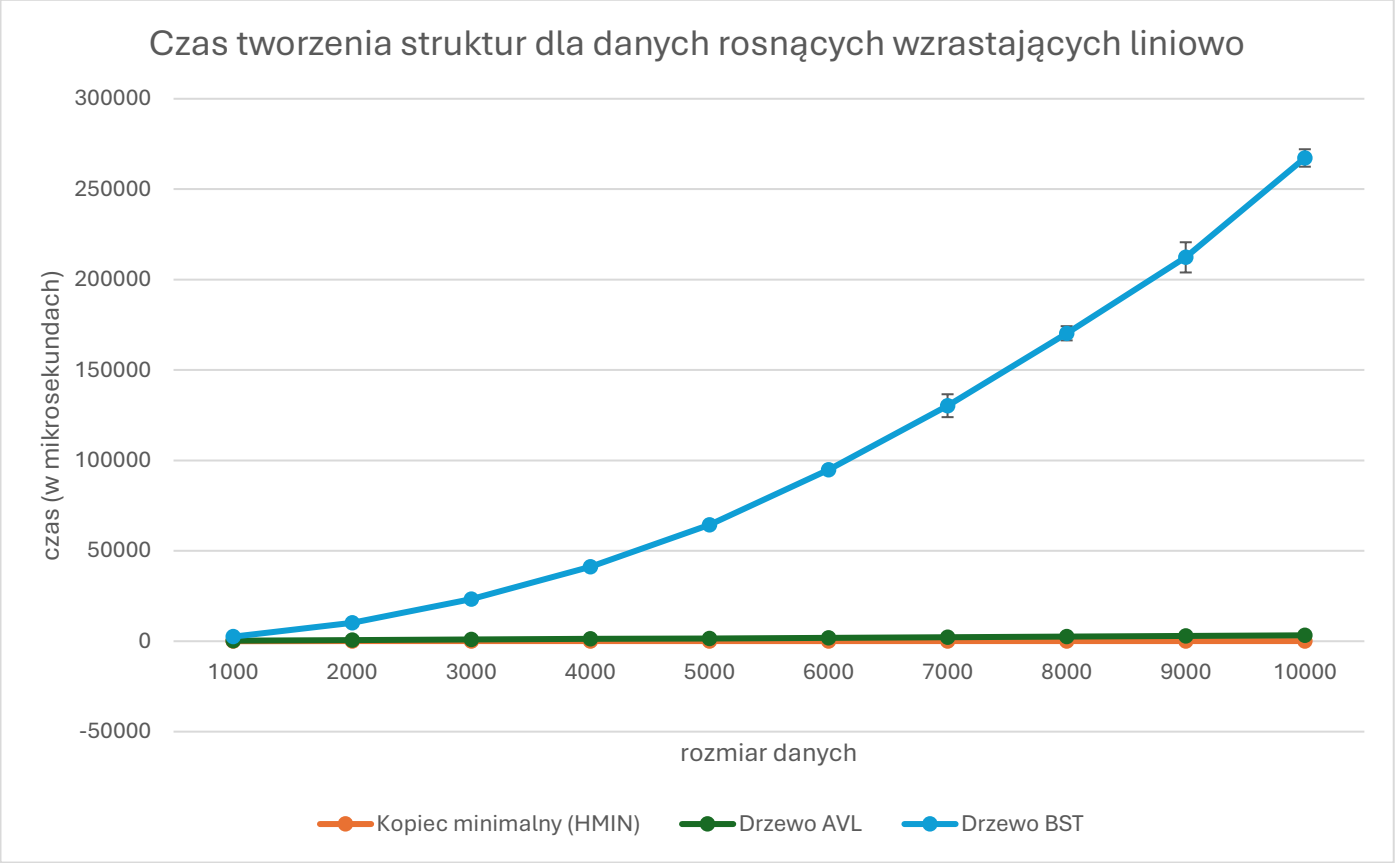
b. Dla danych malejących:

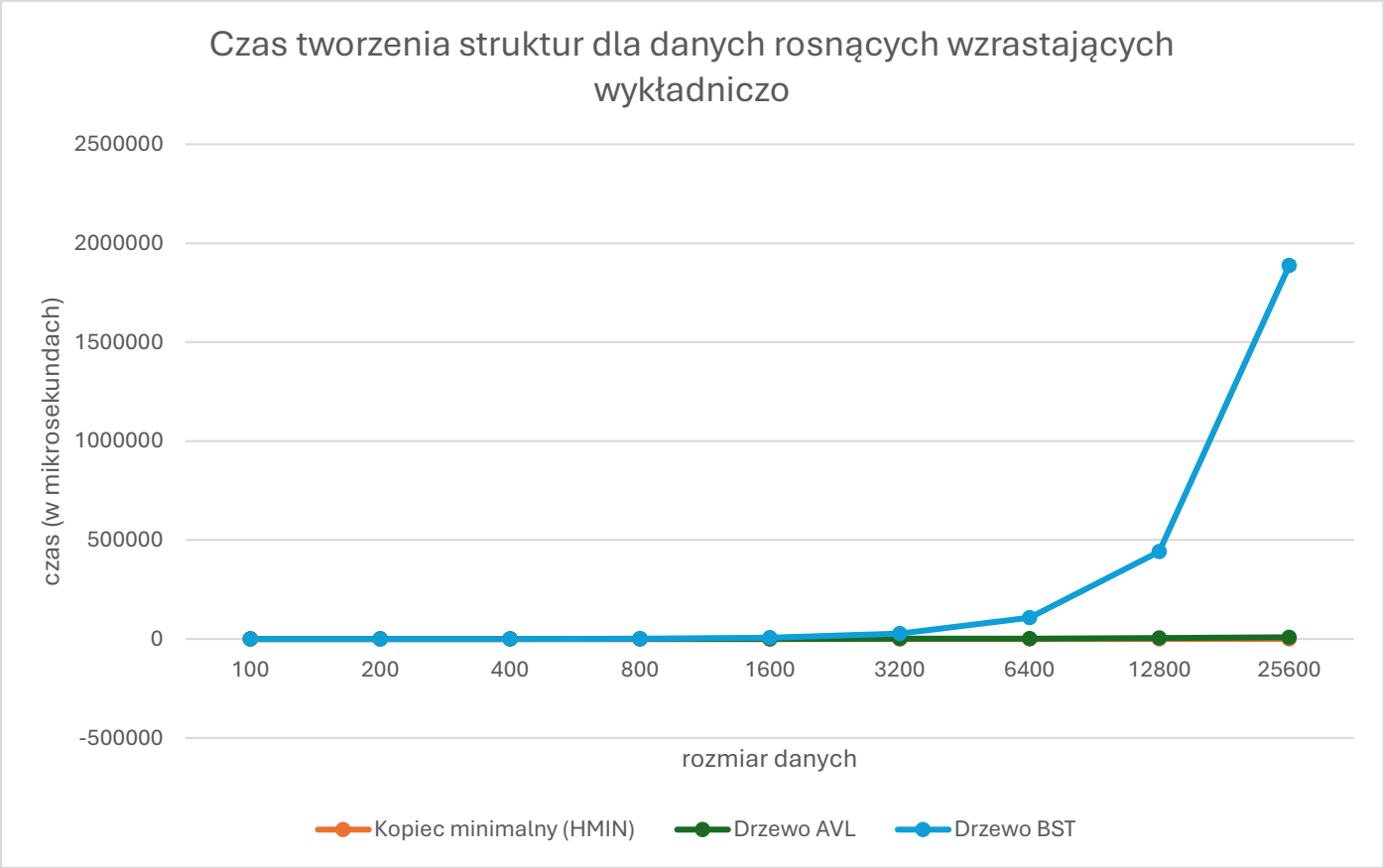




Przy danych malejących już widzimy lekką zmianę w porównaniu do danych losowych. Zarówno kopiec minimalny jak i drzewo AVL zachowują bardzo wysoką wydajność, porównywalną do danych losowych. Metoda bisekcji w AVL skutecznie chroni przed wpływem uporządkowania danych. Przy drzewie BST następuje drastycznie załamanie wydajności. Czas tworzenia rośnie nieliniowo i drzewo wstawiane metodą FCFS przy danych posortowanych malejąco degeneruje się w listę, co zmienia złożoność z O(n logn) na O(n^2).

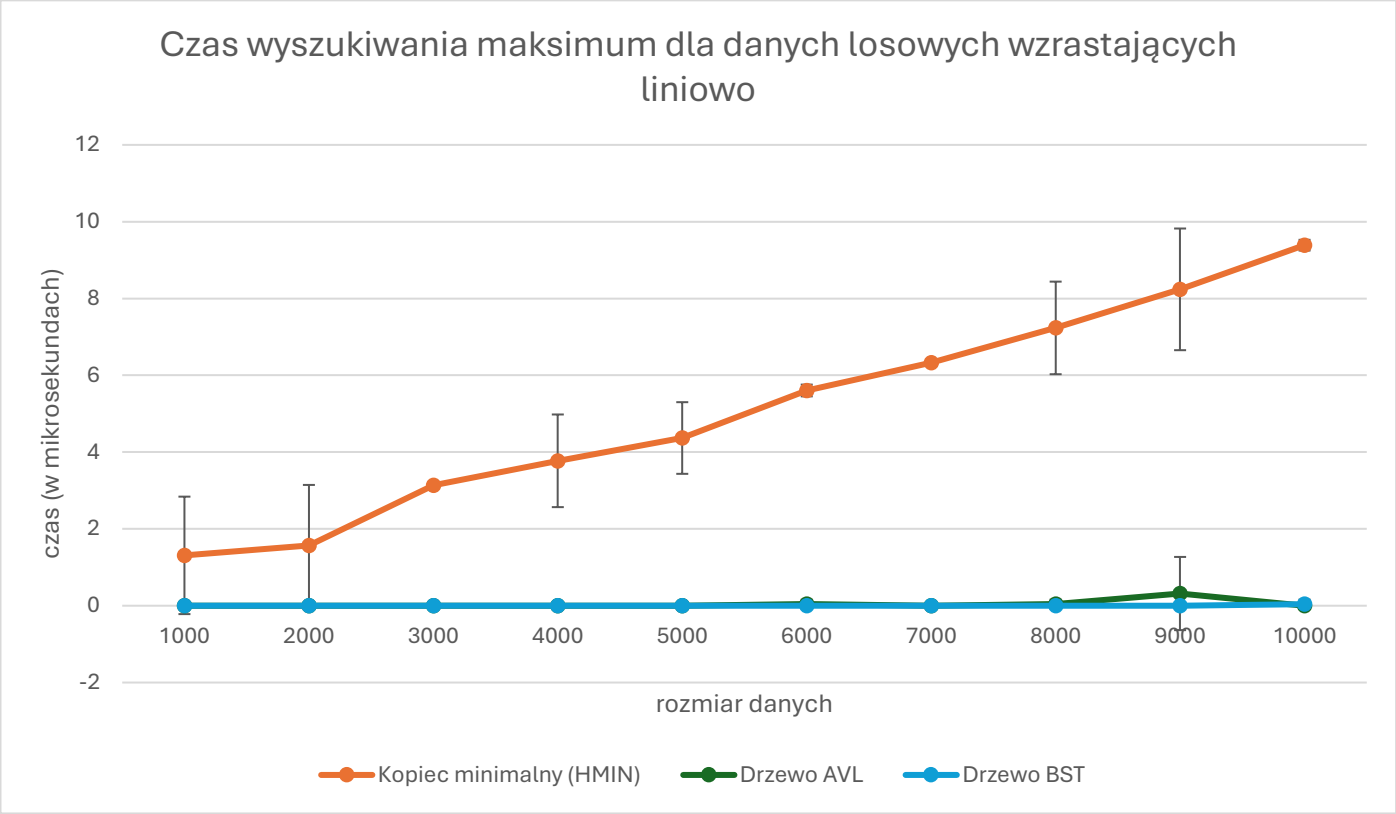
c. Dla danych rosnących:

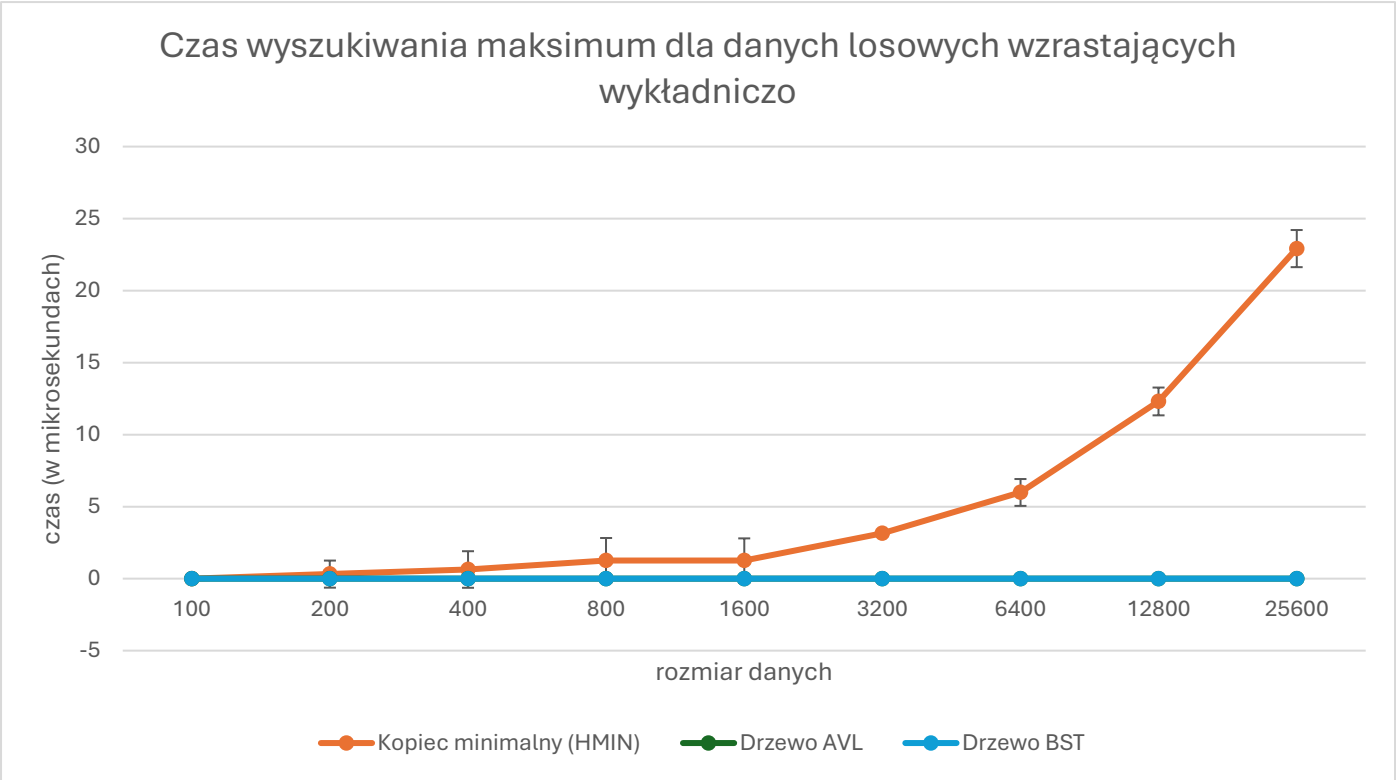




Podobnie jak przy danych malejących kopiec minimalny oraz drzewo AVL wykazują niemal całkowitą odporność na uporządkowanie danych, utrzymując bardzo niskie czasy operacji. Przy drzewie BST odnotowano jedne z najgorszych wyników w eksperymencie. Dane rosnące powodują maksymalną degenerację drzewa w prawostronną ścieżkę, co czyni kolejny proces wstawiania elementu skrajnie kosztownym.

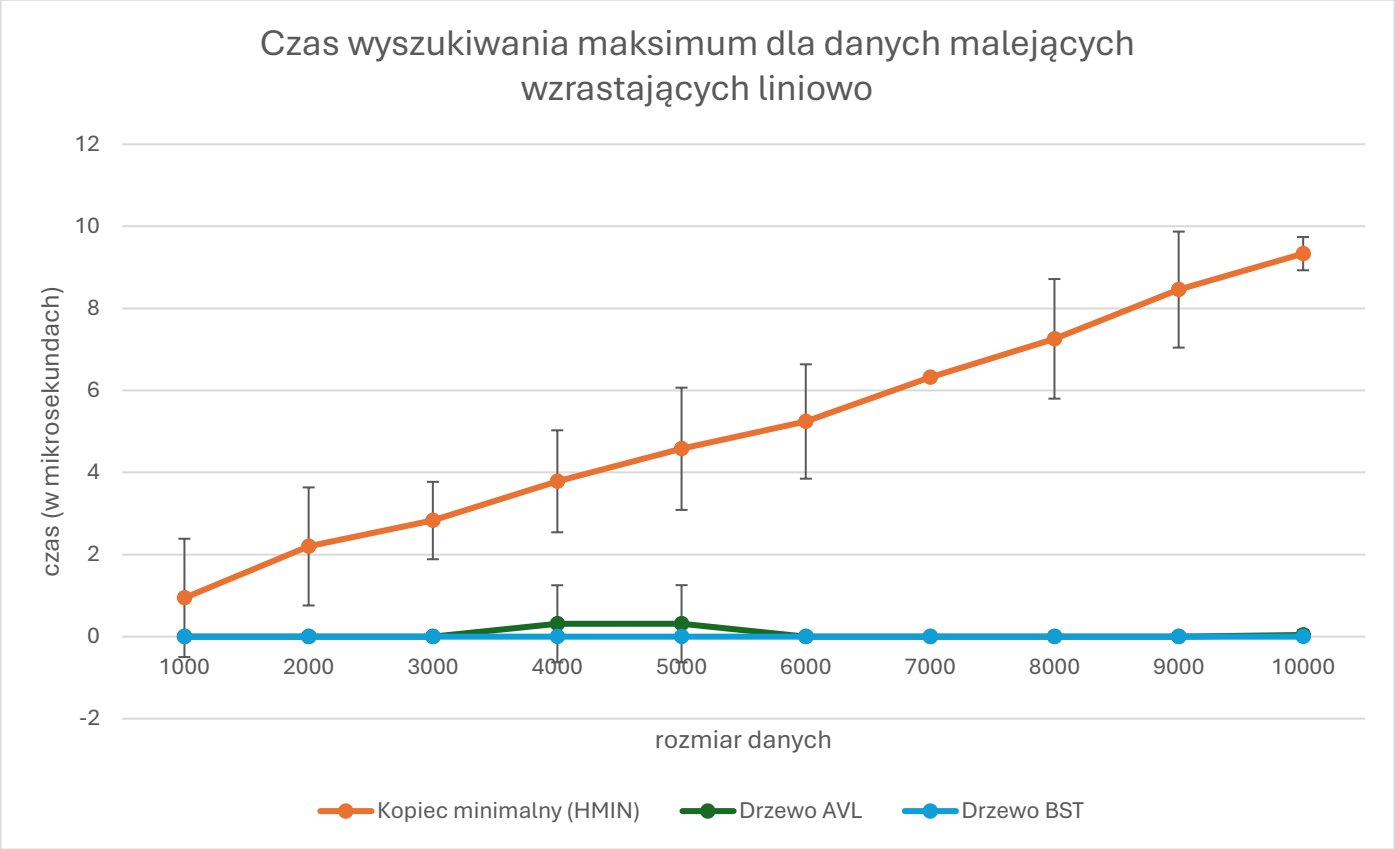
5. **Analiza wyników** – dla każdego typu danych wymienionych w punkcie 3 porównanie czasu wyszukiwania maksimum w strukturze.
    a. Dla danych losowych:

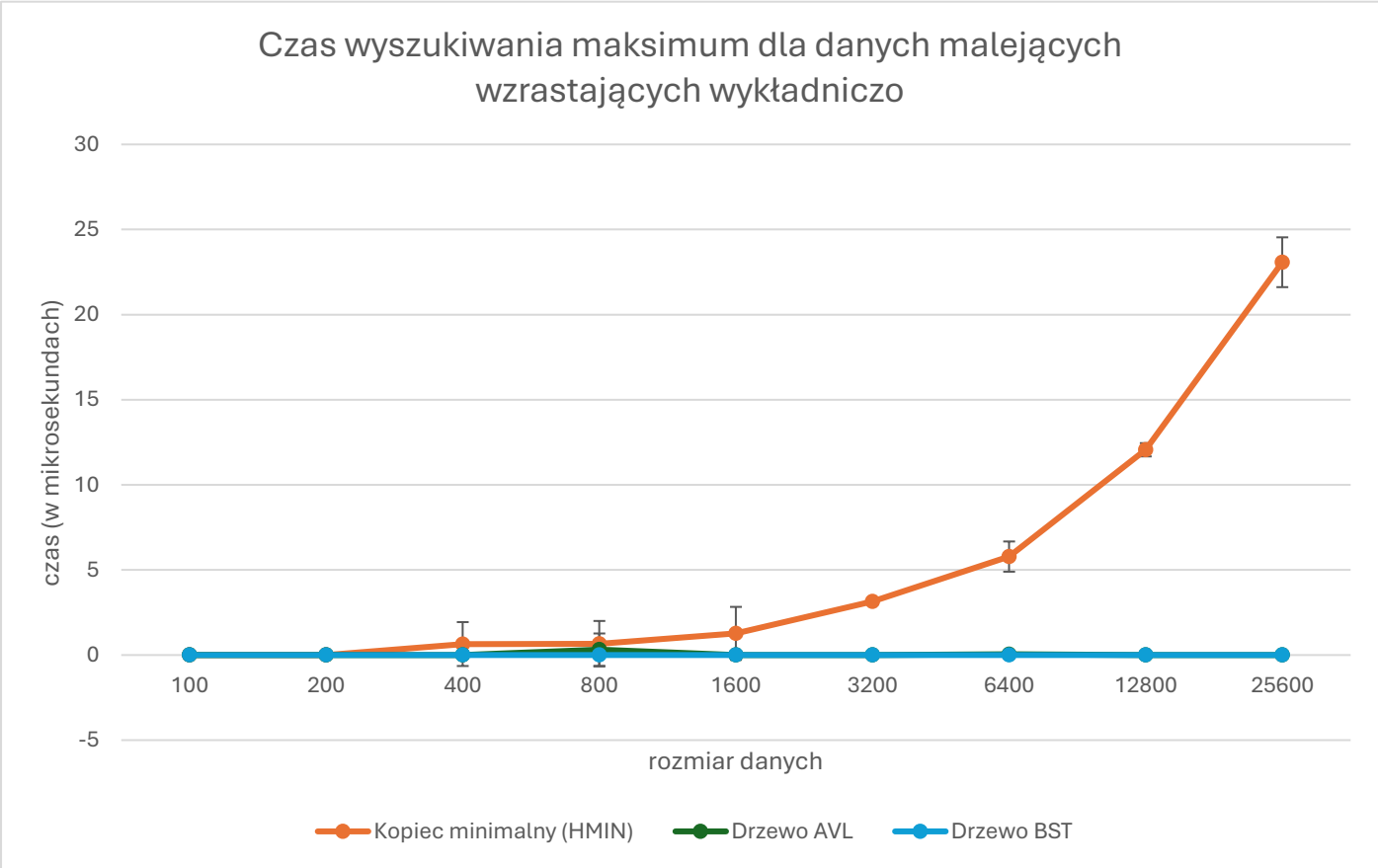




Wyszukiwanie maksimum przy danych losowych dla drzewa BST oraz drzewa AVL jest błyskawiczne, ponieważ polega na przejściu po prawej krawędzi drzewa, która w przypadku danych losowych ma wysokość logarytmiczną. Za to szukanie maksimum w kopcu minimalnym jest wyraźnie wolniejsze od drzew na obu wykresach. W kopcu największy element znajduje się w jednym z liści, co zmusza algorytm do przeszukiwania dolnej warstwy struktury zamiast prostej ścieżki.

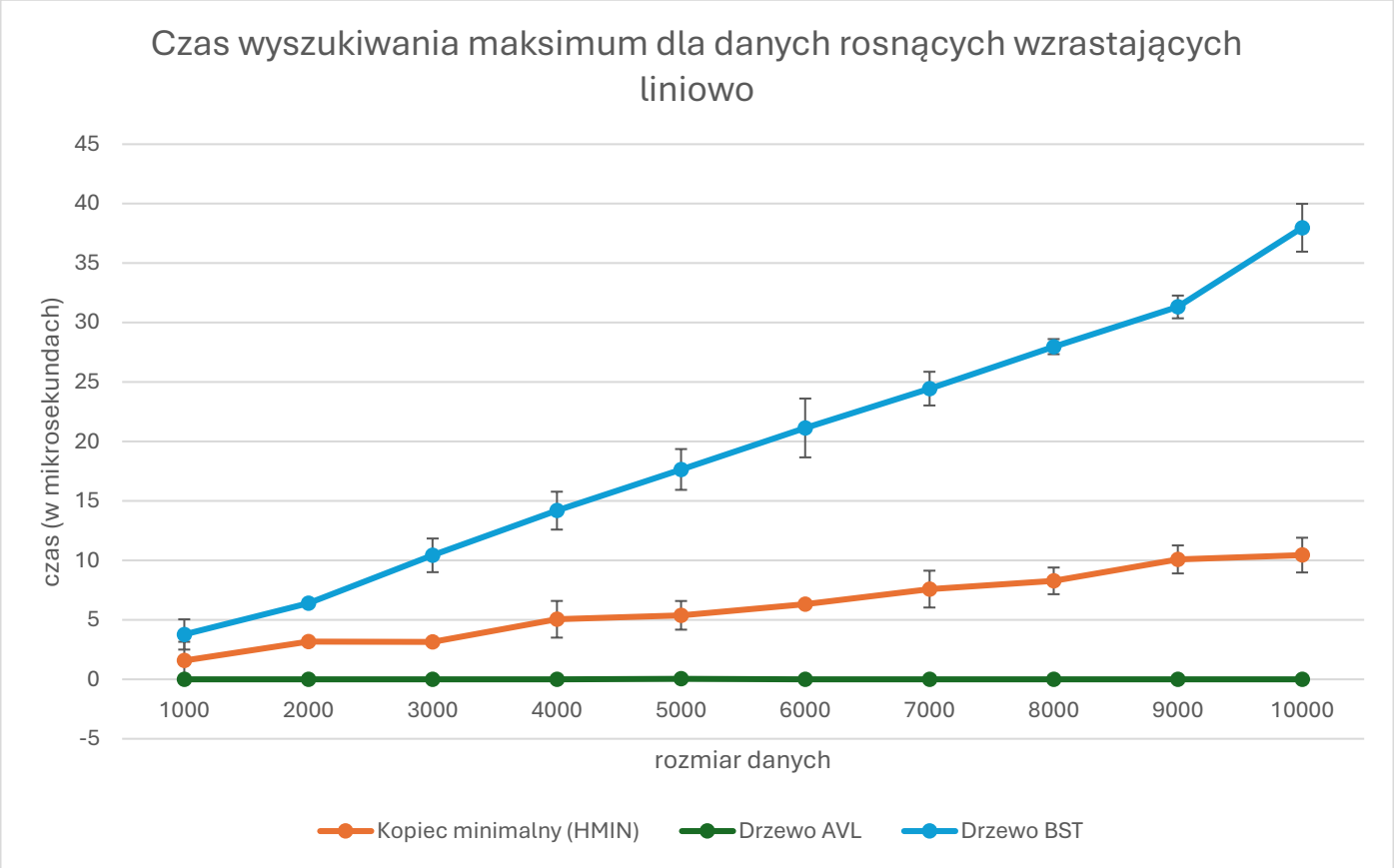
b. Dla danych malejących:

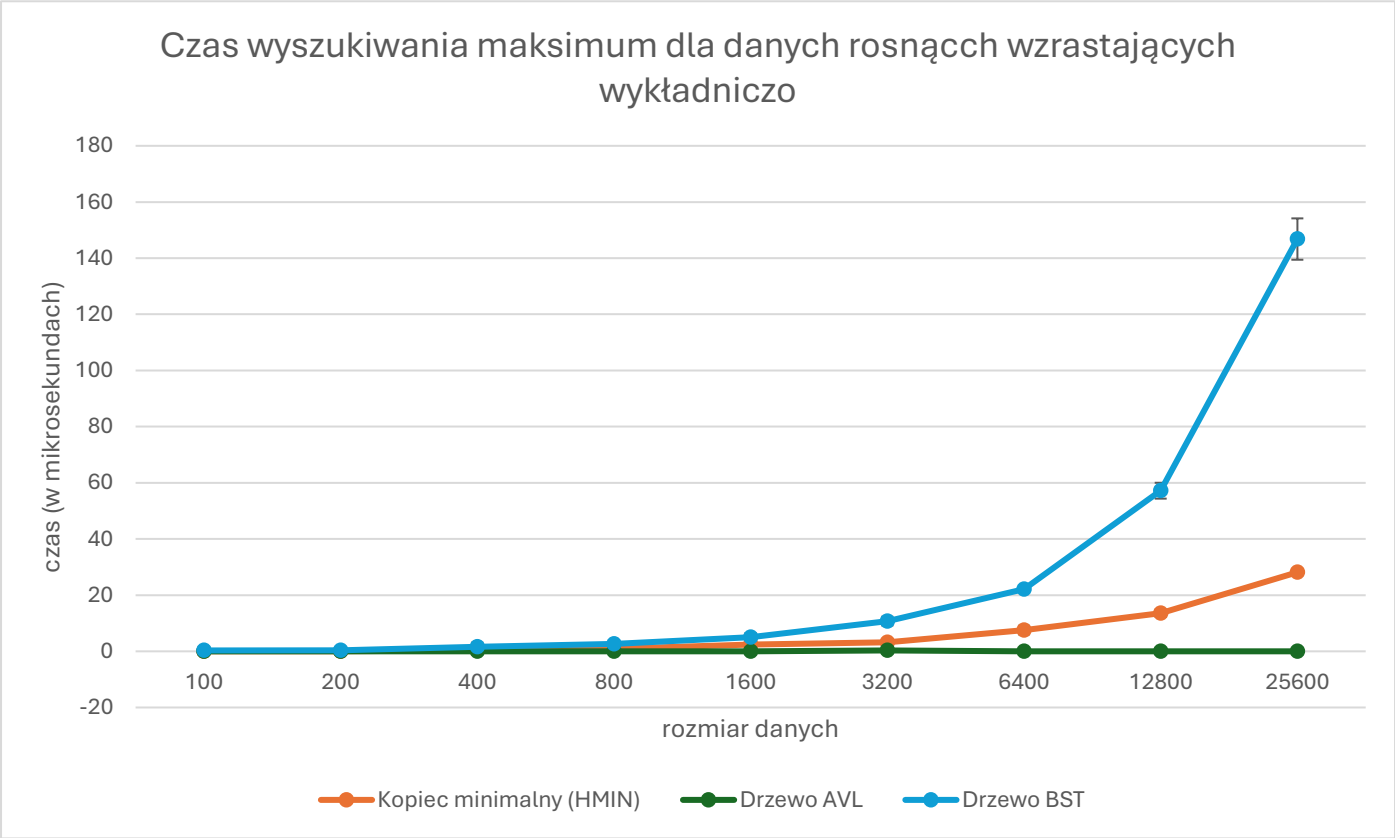




Przy danych malejących ponownie drzewa BST oraz AVL wykazują rekordową szybkość. W przypadku BST zdegenerowanego „w lewo”, maksimum znajduje się w samym korzeniu, co teoretycznie skraca czas wyszukiwania do O(1). Czas wyszukiwania w kopcu minimalnych rośnie liniowo wraz z rozmiarem danych, co potwierdza jego gorszą przydatność do operacji wyszukiwania elementów innych niż minimalny.

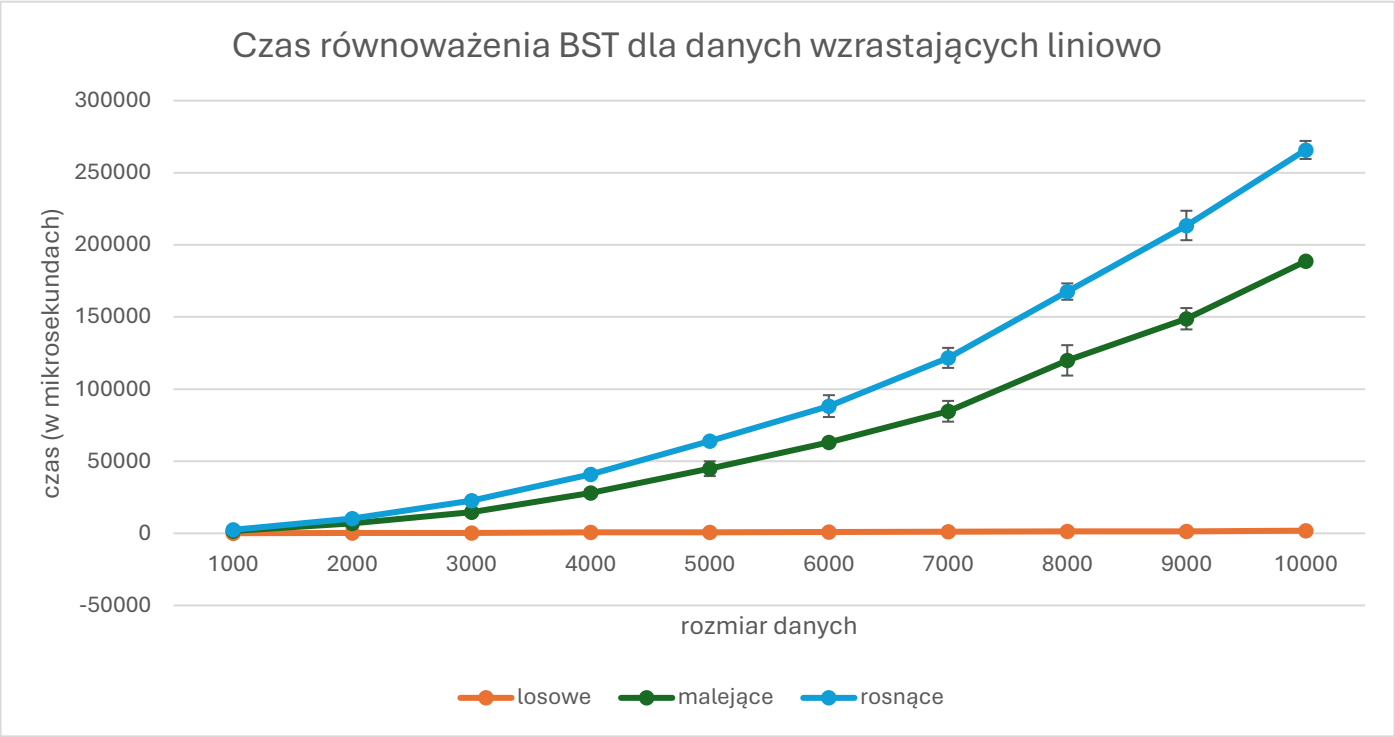
c. Dla danych rosnących:

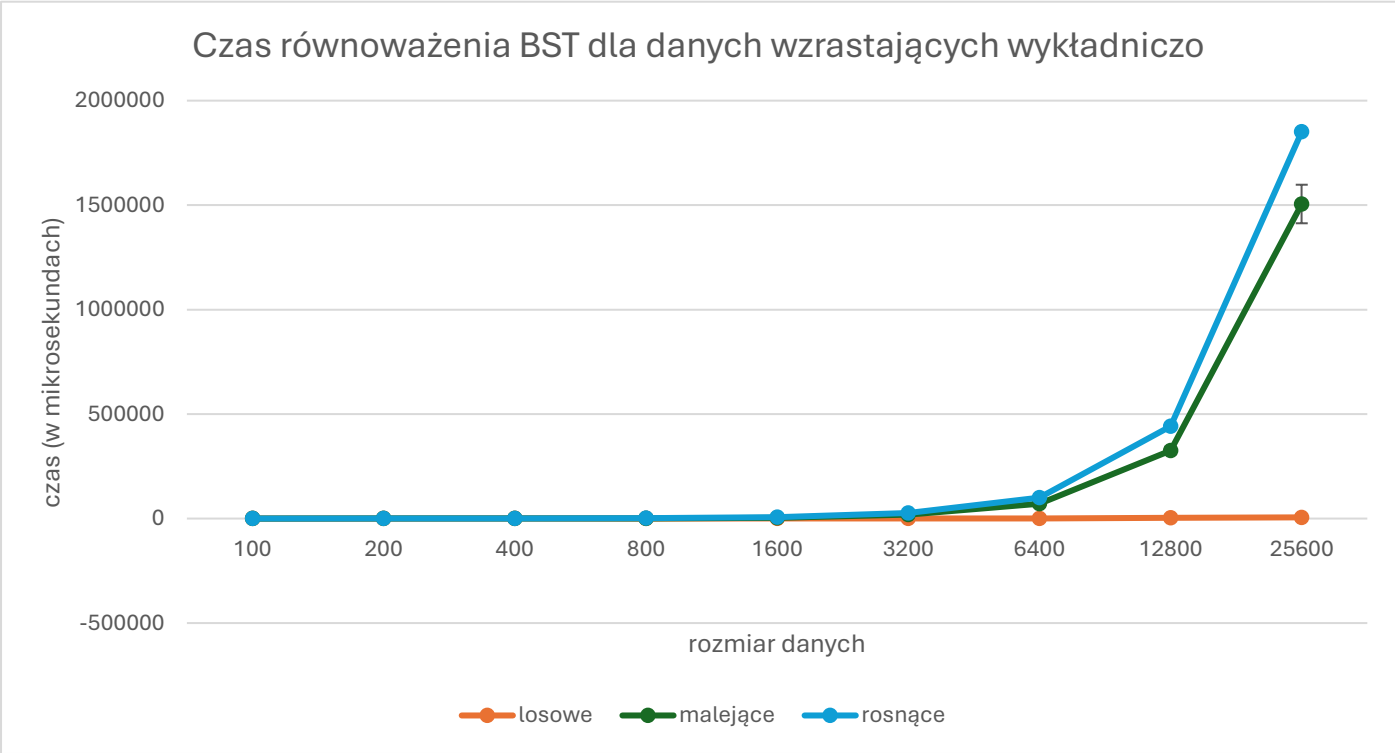




Dla danych rosnących na obu wykresach widać zmianę w porównaniu do danych losowych i malejących. Drzewo AVL nadal pozostaje bardzo efektywne dzięki stałemu utrzymywaniu logarytmicznej wysokości drzewa. Czas wyszukiwania dla kopca minimalnego rośnie stabilnie, za to drzewo BST wykazuje najgorszą charakterystykę. W zdegenerowanym drzewie szukanie maksimum wymaga przejścia przez wszystkie węzły od korzenia do ostatniego liścia, co zmienia złożoność wyszukiwania z O(log n) na O(n).

6. **Analiza wyników** – porównanie czasu operacji równoważenia drzewa BST dla danych losowych, malejących i rosnących:





Jak widać na wykresach równoważenie drzewa BST dla danych losowych jest relatywnie szybkie. Wynika to z faktu, że drzewa budowane z losowych kluczy są zazwyczaj naturalnie „niskie" i zbliżone do drzew zrównoważonych, więc algorytm ma mniej pracy przy reorganizacji węzłów. Przy danych malejących oraz rosnących czas operacji znacząco wzrasta w porównaniu do danych losowych. Dla danych malejących wynika to z faktu, że drzewo wyjściowe jest całkowicie zdegenerowane (jedna długa ścieżka w lewo), co wymusza na algorytmie wykonanie pełnej sekwencji rotacji lub całkowitą przebudowę struktury od podstaw. A dla danych rosnących mamy przypadek pesymistyczny – takie dane tworzą drzewo w postaci długiej „prawostronnej" listy. Koszt dojścia do poszczególnych elementów w celu ich reorganizacji jest maksymalny, co sprawia, że proces przywracania balansu jest najbardziej kosztowny obliczeniowo.

7. Analiza porównawcza obu eksperymentów

Na podstawie zgromadzonych danych pomiarowych przeprowadzono porównanie wyników uzyskanych w eksperymencie 1 (wzrost liniowy) oraz eksperymencie 2 (wzrost wykładniczy).

- **Widoczność różnic między strukturami:** Różnice między strukturami są zdecydowanie bardziej wyraźne w **eksperymencie ze wzrostem wykładniczym**. Przy dużych wartościach n (np. 25 600) różnica między czasem tworzenia kopca (ok. 425 µs), a zdegenerowanego drzewa BST dla danych rosnących (ponad 1 800 000 µs) staje się gigantyczna. Eksperyment liniowy przy mniejszym zakresie n nie uwydatnia tak drastycznie słabości drzewa BST.
- **Zmiana kształtu wykresów przy wzroście wykładniczym:** Skok wykładniczy gwałtownie "wypycha" krzywe o złożoności wyższej niż liniowa (np. kwadratowe $O(n^2)$ dla zdegenerowanego BST) w górę wykresu. Podczas gdy wykresy dla kopca i AVL pozostają relatywnie płaskie, krzywa BST dla danych posortowanych przybiera kształt paraboli, co natychmiast obnaża charakter złożoności asymptotycznej tej struktury.
- **Stabilność wysokości drzewa AVL:** Drzewo AVL utrzymuje stabilną, logarytmiczną wysokość niezależnie od typu danych. Dzięki zastosowaniu metody bisekcji przy budowie oraz mechanizmom równoważenia, struktura ta jest w pełni odporna na charakter danych wejściowych, co potwierdzają bardzo zbliżone czasy operacji dla danych losowych, rosnących i malejących.
- **Wpływ typu danych na czasy operacji:**
    - **Największy wpływ:** Odnotowano w przypadku **drzewa BST**. Typ danych determinuje tu, czy algorytm działa wydajnie (O(n logn)), czy ulega załamaniu wydajnościowemu ($O(n^2)$).

      Z czego to wynika? Wstawianie po kolei posortowanych danych sprawia, że drzewo nie tworzy rozgałęzień (ulega zdegenerowaniu) i przybiera formę wolnej w przeszukiwaniu listy jednokierunkowej.
    - **Najmniejszy wpływ:** Dotyczy **kopca minimalnego (HMIN) oraz drzewa AVL**. Struktury te wykazują dużą odporność na uporządkowanie początkowe danych.

      Z czego to wynika?

      W drzewie AVL: Działa tu wbudowany mechanizm samobalansowania. Po dodaniu elementu algorytm rotuje węzły, siłowo wymuszając optymalny kształt drzewa.

      W kopcu HMIN: Kopiec musi być tzw. drzewem zupełnym (wypełnianym równo, poziom po poziomie). Jego fizyczny kształt zależy wyłącznie od liczby elementów, a nie od ich wartości.

- **Podobieństwo efektów dla danych rosnących i malejących:** Efekty są bardzo zbliżone w przypadku **tworzenia struktury i równoważenia** – w obu wariantach BST degeneruje się do postaci listy. Jednak przy **wyszukiwaniu maksimum** występuje kluczowa różnica:
    - Dla danych **malejących**, maksimum znajduje się w korzeniu, co skraca czas wyszukiwania do O(1).
    - Dla danych **rosnących**, maksimum znajduje się na samym końcu ścieżki, co wymusza przejście przez wszystkie węzły (O(n)).
- **Wpływ typu danych na wysokość drzewa i koszt operacji:** W drzewie BST dane posortowane powodują maksymalną wysokość drzewa, co zmienia strukturę w listę i drastycznie podnosi koszt każdej operacji. W AVL wysokość jest zawsze optymalna, co gwarantuje minimalne i przewidywalne koszty wyszukiwania niezależnie od postaci danych

8. Analiza złożoności obliczeniowej

Złożoność obliczeniowa badanych struktur zależy w głównej mierze od początkowego uporządkowania danych wejściowych oraz specyfiki zaimplementowanych algorytmów. Poniżej przedstawiono teoretyczną analizę czasową dla badanych algorytmów.

Struktura	Operacja	Złożoność średnia (dane losowe)	Złożoność pesymistyczna (dane posortowane)
BST	Tworzenie	O(n log n)	$O(n^2)$
	Wyszukiwanie elementu	O(log n)	O(n)
	Równoważenie	O(n)	O(n)
AVL	Tworzenie	O(n log n)	O(n log n)
	Wyszukiwanie elementu	O(log n)	O(log n)
HMIN	Tworzenie	O(n)	O(n)
	Wyszukiwanie elementu	O(n)	O(n)

8.1 Porównanie wyników teoretycznych z eksperymentalnymi i rozbieżności

- Wyniki uzyskane w eksperymencie w większości potwierdzają założenia teoretyczne zawarte w tabeli (w tym drastyczny wzrost czasu tworzenia BST dla danych posortowanych, co udowadnia degenerację struktury i czas $O(n^2)$). Możemy zaobserwować jednak dwie interesujące rozbieżności:

- Główna rozbieżność (Wyszukiwanie w BST): Teoretycznie szukanie elementu w zdegenerowanym (rozciągniętym w jedną linię) drzewie BST to przypadek pesymistyczny O(n). Jednak podczas pomiarów wyszukiwania **maksimum** w danych posortowanych malejąco, czas ten spadł praktycznie do zera (O(1)). Wynika to z faktu, że pierwsza wstawiana liczba była zarazem największa, więc od razu stawała się głównym korzeniem, całkowicie eliminując konieczność przeszukiwania drzewa.

- Mniejsza rozbieżność (Tworzenie Kopca HMIN): Mimo że teoretyczny czas budowy kopca to zawsze optymalne O(n), eksperyment wykazał, że budowa z danych posortowanych rosnąco jest minimalnie szybsza. Dzieje się tak, ponieważ ciąg rosnący z definicji przypomina już kopiec minimalny – algorytm wykonuje wtedy głównie sprawdzenia, niemal całkowicie omijając kosztowne czasowo zamiany komórek w pamięci.

- **Krytyczna rozbieżność (Równoważenie zdegenerowanego BST):** Choć teoretyczna złożoność pesymistyczna równoważenia drzewa BST wynosi O(n), eksperyment ze wzrostem wykładniczym ujawnił, że w przypadku danych posortowanych (drzewo całkowicie zdegenerowane) czas wykonania algorytmu rośnie kwadratowo – **$O(n^2)$**. Doskonale widać to w tabeli wyników dla danych malejących: przy podwojeniu rozmiaru n z 12 800 do 25 600, czas operacji nie uległ podwojeniu, lecz wzrósł ponad czterokrotnie (z ok. 325 618 µs do 1 505 359 µs). Taka rozbieżność wynika najprawdopodobniej z zastosowanej implementacji (np. ukrytych kosztów ponownego przeszukiwania wskaźników od korzenia przy operacjach rotacji na bardzo długiej liście), a nie z samej definicji algorytmu równoważenia.

8.2 Czy eksperymentalna złożoność czasowa zależy od typu danych wejściowych?

- Tak, złożoność eksperymentalna zależy od ułożenia danych początkowych, ale siła tego wpływu jest drastycznie różna w zależności od struktury:
- Drzewo BST (Zależność krytyczna): Typ danych decyduje tu o wszystkim. To on determinuje, czy algorytm zadziała wydajnie (O(n log n) dla danych losowych), czy jego wydajność ulegnie całkowitemu załamaniu ($O(n^2)$ dla danych posortowanych).
- Drzewo AVL oraz Kopiec (Zależność marginalna): Te struktury są uodpornione na typ danych. Rodzaj ciągu początkowego nie zmienia ich klasy złożoności – wykresy czasowe rosną w tym samym, stałym tempie, a widoczne w eksperymencie różnice rzędu mikrosekund wynikają jedynie z innej liczby mikrokroków wewnątrz pętli (np. mniejszej liczby rotacji w AVL).

8.3 Które przypadki odpowiadają złożoności średniej, a które pesymistycznej?

- Biorąc pod uwagę architekturę badanych struktur (ze szczególnym uwzględnieniem braku samorównoważenia w zwykłym BST), przypadki klasyfikujemy następująco:
- Przypadek średni– dane losowe: Brak uporządkowania jest w tym wypadku zjawiskiem pożądanym. Elementy rozkładają się naturalnie i chaotycznie na lewe oraz prawe poddrzewa, co zapobiega degeneracji struktury i pozwala na utrzymanie wydajnych, logarytmicznych czasów operacji.
- Przypadek pesymistyczny (najgorszy) – Dane posortowane (rosnące i malejące): Podawanie posortowanych ciągów do struktury BST to scenariusz najgorszy z możliwych. Drzewo staje się skrajnie asymetryczne, redukując się do zwykłej listy jednokierunkowej, co drastycznie wydłuża czasy wstawiania i przeszukiwania do maksimum.

9. Wnioski końcowe:

- **Najbardziej efektywna struktura w praktyce:**
  Najbardziej stabilną i wszechstronną strukturą okazało się **Drzewo AVL**. Choć Kopiec (HMIN) teoretycznie oferuje najszybszy czas budowy (O(n)), to w ogólnym rozrachunku strukturą najbardziej wydajną jest AVL. Wynika to z faktu, że czas wyszukiwania w AVL jest logarytmiczny (O(log n)), co przy dużych zbiorach danych pozwala na znalezienie elementu tysiące razy szybciej niż w Kopcu (O(n)).

- **Struktura najbardziej wrażliwa na postać danych wejściowych:**
  Bezsprzecznie jest to zwykłe **Drzewo BST (FCFS)**. Eksperyment udowodnił, że pozbawienie drzewa mechanizmów samorównoważących czyni je całkowicie bezbronnym wobec złośliwych danych. Podanie posortowanych ciągów wejściowych doprowadziło do katastrofalnego załamania wydajności – czas operacji wzrósł do wartości kwadratowych i liniowych.

- **Porównanie zachowania drzewa zrównoważonego (AVL) i niezrównoważonego (BST):**
  Zestawienie tych dwóch struktur idealnie ukazuje, dlaczego w profesjonalnej informatyce stosuje się wyłącznie drzewa zrównoważone. Niezrównoważone BST działa poprawnie tylko wtedy, gdy ma "szczęście" otrzymać idealnie wymieszane dane losowe. Drzewo zrównoważone (AVL) jest z kolei całkowicie odporne na kształt danych – zawsze wymusza na sobie optymalną wysokość i minimalizuje najdłuższe ścieżki poszukiwań. Zrównoważenie jest absolutnym fundamentem przewidywalności systemów komputerowych.

- **Ocena sposobu skalowania danych (Wzrost liniowy vs Wykładniczy):**
  Zdecydowanie to **skalowanie wykładnicze** (np.: 1000, 2000, 4000, 8000...) lepiej ujawnia rzeczywisty charakter złożoności asymptotycznych. Przy małych porcjach i wzroście liniowym, wykresy funkcji liniowych O(n) oraz logarytmiczno-liniowych O(n log n) mogą wizualnie układać się w niemal identyczne proste. Skok wykładniczy błyskawicznie obnaża naturę algorytmu, gwałtownie "wypychając" krzywe kwadratowe (takie jak zdegenerowane BST) w górę wykresu i wyraźnie oddzielając je od struktur stabilnych (Kopiec, AVL)

TABELKI POMIARÓW

Tworzenie struktur dla danych wzrastających liniowo:

1. **Dane losowe:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	9,44934	354,3106	158,3525
2000	25,53966	722,8388	314,3725
3000	47,16576	1119,381	628,4565
4000	57,81794	1496,414	698,0295
5000	84,0963	1895,856	727,5915
6000	96,2227	2337,889	835,865
7000	94,3946	2734,811	1013,631
8000	130,0251	3115,058	1180,28
9000	127,9912	3549,431	1340,026
10000	176,6848	3953,021	1552,8

   b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	14,43448	11,50157	316,7192
2000	12,81431	13,56133	385,0385
3000	15,47313	15,12063	314,2475
4000	11,4766	15,17735	235,0577
5000	42,93999	39,20572	46,63584
6000	10,80995	11,36446	45,80944
7000	77,0731	76,08674	367,3191
8000	74,5984	67,13389	303,0568
9000	64,28016	81,92217	363,2701
10000	45,99503	19,10598	39,78881

2. **Dane malejące:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	12,62226	303,0896	1656,305
2000	20,99294	612,6894	6926,046
3000	47,24414	946,8195	15404,66
4000	60,22762	1240,614	29588,25
5000	45,1204	1575,253	42853,33
6000	73,0253	1932,101	63115,71
7000	70,1654	2239,986	88214,71
8000	78,6133	2542,496	119886,6
9000	110,3599	2952,41	148097,6
10000	140,4495	3303,654	189366,2

b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	15,46044	15,60271	247,1988
2000	14,01276	15,61571	588,2235
3000	15,66677	10,05414	788,3559
4000	9,235449	16,25911	3418,203
5000	62,00181	16,17833	413,2565
6000	65,91991	64,49641	557,5278
7000	74,99431	62,16881	7689,09
8000	78,61351	44,47482	7777,586
9000	72,25809	68,28758	7575,687
10000	47,28473	44,84056	480,7384

3. Dane rosnące:

a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	6,28092	303,5422	2514,45
2000	9,4643	621,7996	10266,82
3000	9,39544	948,2332	23254,07
4000	18,9931	1242,314	41082,73
5000	8,0086	1589,826	64416,76
6000	80,6285	1896,723	94843,04
7000	31,6773	2216,165	130262,8
8000	31,8369	2546,016	170310,3
9000	31,3969	2927,973	212293,5
10000	47,5315	3276,035	267264,7

b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	12,56184	15,22856	320,6305
2000	14,45711	14,0889	487,3598
3000	14,35177	9,287663	630,2622
4000	15,51155	13,97703	384,6821
5000	13,28031	21,01396	387,6873
6000	76,83397	19,62782	707,3498
7000	63,35528	5,248945	6328,982
8000	46,1631	17,42824	3934,141
9000	62,79386	74,81431	8356,692
10000	72,61154	75,16265	4823,044

Wyszukiwanie maksimum dla danych wzrastających liniowo:

1. **Dane losowe:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	1,31113	0	0
2000	1,572542	0	0
3000	3,13628	0	0
4000	3,77373	0	0
5000	4,367374	0	0
6000	5,604958	0,04004	0
7000	6,331688	0	0
8000	7,233842	0,04005	0
9000	8,23827	0,317684	0
10000	9,388734	0	0,04004

   b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	1,529056	0	0
2000	1,572672	0	0
3000	0,007776	0	0
4000	1,205963	0	0
5000	0,932983	0	0
6000	0,153391	0,12012	0
7000	0,041439	0	0
8000	1,205558	0,12015	0
9000	1,584285	0,953052	0
10000	0,139245	0	0,12012

2. **Dane malejące:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	0,943824	0	0
2000	2,19795	0	0
3000	2,829734	0	0
4000	3,785488	0,313072	0
5000	4,578034	0,313628	0
6000	5,242998	0	0
7000	6,319372	0	0
8000	7,257334	0	0
9000	8,45747	0	0
10000	9,333328	0,040052	0

b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	1,441728	0	0
2000	1,439	0	0
3000	0,943584	0	0
4000	1,24321	0,939216	0
5000	1,489504	0,940884	0
6000	1,393691	0	0
7000	0,029881	0	0
8000	1,457894	0	0
9000	1,413744	0	0
10000	0,405063	0,120156	0

3. **Dane rosnące:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	1,577708	0	3,77288
2000	3,169318	0	6,397988
3000	3,144596	0	10,42687
4000	5,048522	0	14,18548
5000	5,381266	0,040042	17,6479
6000	6,30478	0	21,13695
7000	7,58645	0	24,4467
8000	8,280516	0	27,97713
9000	10,08329	0	31,31652
10000	10,4458	0	37,97476

b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
1000	1,577915	0	1,271775
2000	0,028524	0	0,227054
3000	0,027688	0	1,414743
4000	1,54321	0	1,593208
5000	1,2059	0,120126	1,715292
6000	0,04496	0	2,472373
7000	1,548843	0	1,417491
8000	1,121984	0	0,636385
9000	1,17792	0	0,957033
10000	1,459849	0	2,011265

Tworzenie struktur dla danych wzrastających wykładniczo:

1. **Dane losowe:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	31,81536	0
200	3,18766	63,14274	0
400	3,1408	132,9656	0
800	9,44352	284,0739	0
1600	25,30726	580,1934	157,6885
3200	40,98938	1202,366	553,6435
6400	127,9287	2491,015	785,311
12800	204,0537	5174,328	2127,608
25600	425,9251	10643,45	4656,903

   b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	0,389248	0
200	9,56298	0,366559	0
400	9,4224	11,97373	0
800	14,42534	0,605492	0
1600	12,65588	14,65643	315,383
3200	14,46249	10,67339	362,4944
6400	64,16938	63,49194	2,427776
12800	69,10037	59,9583	358,6853
25600	71,45766	85,87158	228,3987

2. **Dane malejące:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	31,35664	0
200	0	57,08076	157,0755
400	3,14114	123,1424	236,2365
800	9,45676	250,1964	1022,359
1600	22,05546	495,3473	4338,971
3200	41,20596	1014,318	17923,05
6400	96,2337	2075,661	72461,74
12800	164,844	4248,469	328266,8
25600	381,1327	8643,922	1757824

b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	0,932741	0
200	0	12,60618	314,1514
400	9,42342	9,562608	360,8599
800	14,44561	9,57237	386,8692
1600	14,43927	14,33048	393,8288
3200	14,44781	8,620804	1339,673
6400	58,71788	30,88789	5470,329
12800	8,332961	18,53265	7129,006
25600	77,69001	102,793	396193,3

3. **Dane rosnące:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	25,93374	78,914
200	0,40126	56,48324	200,863
400	3,14866	117,452	463,3225
800	0	246,1727	1812,372
1600	3,13604	505,1222	6835,23
3200	6,26828	1021,84	26702,17
6400	16,1357	2063,959	107053,3
12800	67,5592	4246,486	440896,5
25600	126,9106	8644,771	1887155

b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	11,33511	236,742
200	1,20378	12,86138	309,2779
400	9,44598	14,93388	379,1919
800	0	12,62307	363,5016
1600	9,40812	0,743069	340,4929
3200	12,53657	13,50815	795,9834
6400	48,4071	47,80309	6277,284
12800	66,23282	49,02216	9357,385
25600	63,47827	67,52956	22220,24

Wyszukiwanie maksimum dla danych wzrastających wykładniczo:

1. **Dane losowe:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	0	0
200	0,313462	0	0
400	0,634894	0	0
800	1,268292	0	0
1600	1,256458	0	0
3200	3,146824	0	0
6400	5,983732	0	0
12800	12,30438	0	0
25600	22,91794	0	0

   b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	0	0
200	0,940386	0	0
400	1,269846	0	0
800	1,553473	0	0
1600	1,538844	0	0
3200	0,011935	0	0
6400	0,932827	0	0
12800	0,965222	0	0
25600	1,28903	0	0

2. **Dane malejące:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	0	0
200	0	0	0
400	0,64512	0	0
800	0,666182	0,31659	0
1600	1,27138	0	0
3200	3,14496	0	0
6400	5,783136	0,04004	0
12800	12,06045	0	0
25600	23,07833	0	0

b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	0	0
200	0	0	0
400	1,29024	0	0
800	1,335041	0,94977	0
1600	1,557196	0	0
3200	0,023793	0	0
6400	0,887723	0,12012	0
12800	0,39072	0	0
25600	1,463228	0	0

3. **Dane rosnące:**
   a. Średni czas w mikrosekundach:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	0	0,313502
200	0,050212	0	0,35746
400	0,313668	0	1,575054
800	0,958476	0	2,613514
1600	2,451996	0	5,087668
3200	3,175166	0,313346	10,76577
6400	7,573146	0	22,17546
12800	13,56774	0	57,19036
25600	28,14268	0	146,7982

b. Odchylenie standardowe:

N	Kopiec minimalny (HMIN)	Drzewo AVL	Drzewo BST
100	0	0	0,940506
200	0,120808	0	0,946185
400	0,941004	0	1,575084
800	1,464168	0	1,337348
1600	1,185452	0	1,434175
3200	0,034412	0,940038	1,577781
6400	1,543808	0	0,13933
12800	1,268	0	2,82267
25600	0,98015	0	7,362584

Równoważenie drzewa BST:

1. **Dane wzrastające liniowo:**
   a. Średni czas w mikrosekundach:

N	Losowe	Malejące	Rosnące
1000	78,527	1652,452	2438,826
2000	313,179	6998,077	10325,6
3000	235,451	14780,14	22612,16
4000	697,988	28028,99	40844
5000	769,902	44831,16	64050,59
6000	935,6895	62965,41	88213,39
7000	1193,797	84631,68	121642,7
8000	1341,795	119919,7	167616,1
9000	1410,206	148793,5	213440,8
10000	1846,167	188599,7	265810,6

   b. Odchylenie standardowe:

N	Losowe	Malejące	Rosnące
1000	235,581	234,2153	242,8221
2000	383,5648	746,3149	238,3642
3000	359,6612	296,0628	351,5661
4000	236,529	2344,48	495,8819
5000	28,41027	5117,544	654,0472
6000	45,74232	338,7702	7569,407
7000	389,4308	7203,43	6935,595
8000	298,21	10539,99	5675,163
9000	314,1495	7410,479	10202,48
10000	343,8106	1200,492	6266,772

2. **Dane wzrastające wykładniczo:**
   a. Średni czas w mikrosekundach:

N	Losowe	Malejące	Rosnące
100	0	0	0
200	0	158,1275	170,26
400	78,634	236,065	313,78
800	0	1082,931	1712,912
1600	247,5755	4342,681	6471,959
3200	393,464	18458,34	26045,47
6400	865,4245	71548,88	100390,5
12800	3064,531	325618,4	441920,8
25600	5799,381	1505359	1851345

   b. Odchylenie standardowe:

N	Losowe	Malejące	Rosnące
100	0	0	0
200	0	316,2594	316,8401
400	235,902	360,5963	384,3009
800	0	404,8638	332,3112
1600	378,9821	402,5161	307,1196
3200	393,4656	2407,738	406,4771
6400	234,2463	5516,738	7786,055
12800	727,7657	18821,45	8612,332
25600	355,0252	92216,06	10578,06

Algorytmy zakodowano w języku C++.

Sprzęt, na którym zostały wykonane pomiary:

Procesor: AMD Ryzen 7 4800H

RAM: 16GB DDR4 3200MHz

Karta Graficzna: NVIDIA GeForce 1650 4GB

System operacyjny:

Windows 11